

ANALISTA PROGRAMADOR UNIVERSITARIO

3°er AÑO 2026

LEGISLACIÓN Y EJERCICIO PROFESIONAL



TP01 · Legislación y Ejercicio Profesional · Punto 5 — Práctica Profesional Supervisada

Equipo (Grupo 2):

Alessandro Benitez Donato

Matias Gabriel Ortega

Baca Wayar Juan Sebastian

Flores Mauro Sebastian

Leonardo N. Cazon Gomez

Docente: Ing. Gustavo Sosa

Repositorio docente: gustso/ProyectoLyEP2026

Fork del equipo: BenitezDonatoAlessandro123456/ProyectoLyEP2026

INFORME DE ANÁLISIS TÉCNICO Y DEUDA TÉCNICA

Proyecto: Panel de Control de Clientes (ProyectoLyEP2026)

Materia: Práctica Profesional Supervisada — Legislación y Ejercicio Profesional Equipo de Trabajo: Grupo 2

Fecha de Entrega: 13 de Septiembre de 2026

Equipo de Trabajo: Grupo 2

1. RESUMEN EJECUTIVO

El prototipo evaluado constituye una Single Page Application (SPA) desarrollada en React 19 con Vite y Bootstrap que simula la gestión de clientes interactuando con la API pública FakeStoreAPI. Si bien la estructura del proyecto presenta una adecuada separación modular en carpetas (componentes, contextos, hooks, páginas, rutas y servicios), el estado actual del código refleja una deuda técnica considerable y vulnerabilidades críticas de seguridad. Los problemas más graves incluyen el almacenamiento de credenciales en texto plano dentro del bundle de JavaScript, una autorización manipulable mediante el navegador, la exposición de contraseñas de clientes en la interfaz visual, la fragmentación en la capa de acceso a datos, 7 vulnerabilidades reportadas en dependencias y la ausencia total de pruebas automatizadas. El sistema requiere una refactorización integral hacia la seguridad, la consistencia arquitectónica y la cobertura de pruebas antes de considerarse apto para producción.

2. TABLA DE HALLAZGOS Y DEUDA TÉCNICA

ID	Problema Detectado	Dimensión	Impacto	Propuesta de Solución
#1	Autorización manipulable desde el cliente: DetalleCliente.jsx valida permisos leyendo localStorage.getItem("role"), permitiendo escalar privilegios a "Gerencia"	Seguridad / Frontend	Alto	Eliminar la clave role en localStorage y derivar los permisos de usuario exclusivamente desde el estado centralizado en Autorizaciones Context.

	manipulando el navegador.			
#2	Capa de datos fragmentada e inconsistente: ListaClientes.js x y DetalleCliente.js realizan peticiones fetch() crudas con URLs hardcodeadas, omitiendo la capa de servicios.	Arquitectura / Código	Medio	Centralizar las operaciones CRUD (getClientes, getClientePorId, crearCliente, eliminarCliente) dentro de clientesService.js con un cliente HTTP común.
#3	Exposición de contraseñas de clientes: DetalleCliente.js renderiza en pantalla el campo cliente.password en texto plano.	Seguridad / UI	Alto	Eliminar la renderización de contraseñas de clientes en la interfaz de usuario por el principio de mínimo privilegio y normativas OWASP.
#4	Validación deficiente en formularios: FormCliente.jsx no valida formatos ni campos obligatorios, e	UX / Integridad de Datos	Medio	Implementar validaciones estructuradas con expresiones regulares, mensajes dinámicos de

	inyecta valores fijos como password: "1234" y lastname: "-".			error e impedir el envío de datos inválidos.
#5	Ausencia total de pruebas automatizadas: El proyecto no dispone de configuración de testing ni archivos de prueba para verificar autenticación y rutas.	Testing / QA	Medio	Configurar Vitest y React Testing Library para implementar pruebas unitarias e integrales sobre Login.jsx y RutaProtegida.jsx.
#6	Credenciales hardcodeadas en el bundle: autorizaciones Services.js incluye cuentas e iteraciones de contraseña en claro (ej. Admin123), accesibles inspeccionando el código.	Seguridad	Alto	Migrar la autenticación a un backend real en Node.js/Express con hash de contraseñas (bcrypt) y tokens JWT.
#7	Vulnerabilidades severas en dependencias: npm audit reporta 7 vulnerabilidades	Seguridad / DevOps	Alto	Ejecutar npm audit fix, actualizar paquetes a versiones seguras e

	s (6 altas) incluyendo bypass CSRF en react-router y fallas en paquetes de build.			integrar la auditoría de dependencias en el flujo de integración continua.
#8	Métricas e indicadores hardcoded : El Dashboard.jsx muestra contadores estáticos ("Clientes 10 / Gerencia 3 / Soporte 3") desacoplados del estado real de la API.	UX / Frontend	Medio	Calcular contadores de forma dinámica procesando los registros consumidos de la API y el contexto de usuarios.
#9	Errores de Linter y código muerto: Presencia de imports sin uso (useAutorizaciones, Navigate), advertencias de Fast Refresh y ramas inalcanzables en Dashboard.jsx.	Calidad de Código	Medio	Limpiar código muerto, reestructurar la exportación de Context/Provider y resolver todas las advertencias de ESLint.

#10	Ausencia de Code Splitting e i18n: Carga masiva de la suite completa de CSS de Bootstrap en el bundle inicial y etiqueta <html lang="en"> en una app en español.	Performance / Accesibilidad	Bajo	Implementar React.lazy con Suspense para carga perezosa de vistas y ajustar etiquetas de idioma y accesibilidad en el HTML.
-----	---	-----------------------------	------	---

3. MEJORA SELECCIONADA POR PARTICIPANTE

Issue #1: Autorización manipulable en DevTools

- **Rama:** feature/autorizacion-segura
- **Asignado a:** papulodev
- **Justificación técnica:** Se priorizó por constituir una brecha crítica de seguridad (Alto Impacto) que permite la escalada no autorizada de privilegios en el navegador mediante la modificación directa de localStorage.

Issue #2: Centralizar capa de datos y unificar manejo de errores en clientesService.js

- **Rama:** feature/2-centralizar-clientesService
- **Asignado a:** juansebaca
- **Justificación técnica:** Se seleccionó para resolver la duplicación de URLs y la inconsistencia en el consumo de APIs, abstrayendo las llamadas HTTP en un único punto reutilizable para mejorar la mantenibilidad.

Issue #3: Arreglo de exposición de contraseña del cliente

- **Rama:** feature/LeonardoCazon-ExposiciónContraseña
- **Asignado a:** LeonardoCazon
- **Justificación técnica:** Esta mejora elimina una fuga de información de alto impacto, garantizando que el sistema enmascare u omita credenciales confidenciales en la interfaz visual bajo los estándares OWASP.

Issue #4: Falta de validación de campos vacíos y formato en FormCliente.jsx

- **Rama:** feature/validacion-formulario-cliente
- **Asignado a:** Dumbi34
- **Justificación técnica:** Se eligió para asegurar la integridad de los datos antes de ser enviados al servidor, evitando registros corruptos e inyecciones de valores arbitrarios en la aplicación.

Issue #5: Agregar tests con Vitest para Login y RutaProtegida

- **Rama:** feature/tests-login-rutaprotegida
- **Asignado a:** BenitezDonatoAlessandro123456
- **Justificación técnica:** Se abordó para dotar al proyecto de infraestructura de calidad (QA) y cobertura de pruebas automatizadas sobre los módulos críticos de autenticación y protección de rutas.

4. BACKLOG PRIORIZADO PARA FUTURAS ITERACIONES

1. **Corrección de Vulnerabilidades de Dependencias (npm audit fix):** Resolver de manera inmediata los 7 reportes de seguridad en librerías base (incluyendo react-router).
2. **Implementación de Backend Propio (Node.js / Express / Base de Datos):** Sustituir FakeStoreAPI por un servidor propio con persistencia real de registros y operaciones.
3. **Autenticación y Autorización en Servidor (JWT + Bcrypt):** Migrar el esquema de credenciales hardcodeadas a hashing de contraseñas y emisión de tokens seguros.
4. **Manejo Centralizado de Errores y Notificaciones:** Implementar un helper global de peticiones HTTP para capturar fallos de red y desplegar alertas dinámicas (*Toasts*).
5. **Cálculo Dinámico de Métricas en Dashboard:** Sustituir contadores fijos por procesamiento de datos en tiempo real.
6. **Resolución de Errores de Linter y Separación de Providers:** Mover contextos y proveedores a archivos independientes para cumplir con las reglas de Fast Refresh de React.
7. **Optimización de Rendimiento y Carga Diferida:** Aplicar React.lazy + Suspense en el enrutado para optimizar el peso del bundle inicial.
8. **Pipeline de Integración Continua (CI/CD):** Configurar GitHub Actions para la ejecución automática de comandos de construcción (build), linter y suite de tests en cada Pull Request.